1. (教材 2.27)

设商店在 (x_0, y_0), 则总路程为

$$w(x_0, y_0) = \sum_{i=1}^{n} |x_0 - x_i| + |y_0 - y_i| = \sum_{i=1}^{n} |x_0 - x_i| + \sum_{i=1}^{n} |y_0 - y_i|.$$

设 $k = \lceil n/2 \rceil$, 取 $x_0 = k\text{-th}(x_1, \cdots, x_n)$, $y_0 = k\text{-th}(y_1, \cdots, y_n)$ 即可. 利用经典的分治算法可以做到 $\mathcal{O}(n)$.

2. (Forces Between Particles)

令 $a_k = [1 \le k \le n]\frac{1}{k^2}$, $b_k = [0 \le k < n]\frac{1}{(n-k)^2}$ 希望计算

$$F_j = Cq_j \left(\sum_{i<j} q_i a_{j-i} - \sum_{i>j} q_i b_{n+j-i} \right) = Cq_j((q * a)_j - (q * b)_{n+j}),$$

其中 $*$ 为序列的和卷积. 使用 FFT 算法即可 $\mathcal{O}(n \log n)$ 地求出结果.

3. (Median of Medians)

(1) 最坏需要 $T(n) = \Theta(n) + T(n-1) = \Theta(n^2)$ 的时间. 例如 pivot 取 $[1, n, 2, n-1, \cdots, n/2]$.

(2) p 至少大于 $n/14$ 个组中位数, 进而至少大于 $n/14 \times 4 = 2n/7$ 个数. 另一侧同理.

(3) 最坏情况下,

$$T(n) = f(n) + T(n/7) + T(5n/7),$$

不妨设其中 $f(n) \le kn$, k 为正常数. 令常数 $c_0 = 7k$, 归纳地,

$$T(n) \le kn + \frac{c_0}{7}n + \frac{5c_0}{7}n \le k + \frac{6}{7} \cdot 7kn = c_0 n.$$

而显然 $T(n) \ge kn$, 所以 $T(n) = \Theta(n)$.

4. (Draw the Skyline)

```
{L_n} ← sort({L_n}, key = (a_i, b_i))
{ℓ_m} ← {L_i : ∀i ∈ [1 : n], i = n ∨ a_i ≠ a_{i+1}}
S ← List{}
for i = 1 to m do
    while |S| ≥ 2 do
        (x_0, y_0) ← intersection(S_{-1}, ℓ_i)
        (x_1, y_1) ← intersection(S_{-2}, S_{-1})
        if x_1 < x_2 then
            break
        S.pop_back()
    S.push_back(ℓ_i)
output S
```

用标准的维护栈的方法求出凸壳即可, 排序部分 $\mathcal{O}(n\log n)$, 其余 $\mathcal{O}(n)$. (如果一定要用分治算法的话, 也可以对 (a_i, b_i) 归并排序的过程中顺便归并凸壳, 但这与上述算法没有本质区别且更难实现.)

5. (Local Minima of A Chessboard)

算法流程如下, 只返回了 minima 的值, 且 probe 调用时使用的是子矩阵的下标, 不过这些调整起来都是平凡的.

```
function solve(n, m, M) :
    assert n ≤ m ∧ M ∈ ℝ_{n×m}
    if m = 1 then
        return M_11
    k ← m/2
    S ← {(i, j) : i ∈ {1, n} ∨ j ∈ {1, k, k + 1, m}}
    for (i, j) ∈ S do
        if M_ij is unknown then
            probe(M_ij)
    (u, v) ← arg min_{(i,j)∈S}{M_ij}
    if M_uv is local minima then
        return M_uv
    else if v > k then
        return solve(m − k, n, M[1 : n, k + 1 : m]^⊤)
    else
        return solve(k, n, M[1 : n, 1 : k]^⊤)
```

算法探测矩阵的边框 (大多在上层递归中被探测过) 和最中间两列, 并找到其最小值 M_{uv}. 如果 M_{uv} 在已探测的值中已经确定是 minima, 则返回; 否则向 (u, v) 所属的一侧子矩阵递归, 根据不等号的传递性, 子矩阵的 minima 一定是原来矩阵的 minima, 正确性得以保证. 除了初始的 $4n-3$ 的边框探测外, 其余递归其实都只有探测中间列的代价, 在原矩阵行上和在列上都只探测了 $2n+n+\cdots+1=\mathcal{O}(n)$ 个元素.